

基于当前仓库源码 · Qt / C++

通信工具代码结构与协议流程图

客户端 (DataSendServer) × 服务端 (DataReceiveServer)

UDP 数据报

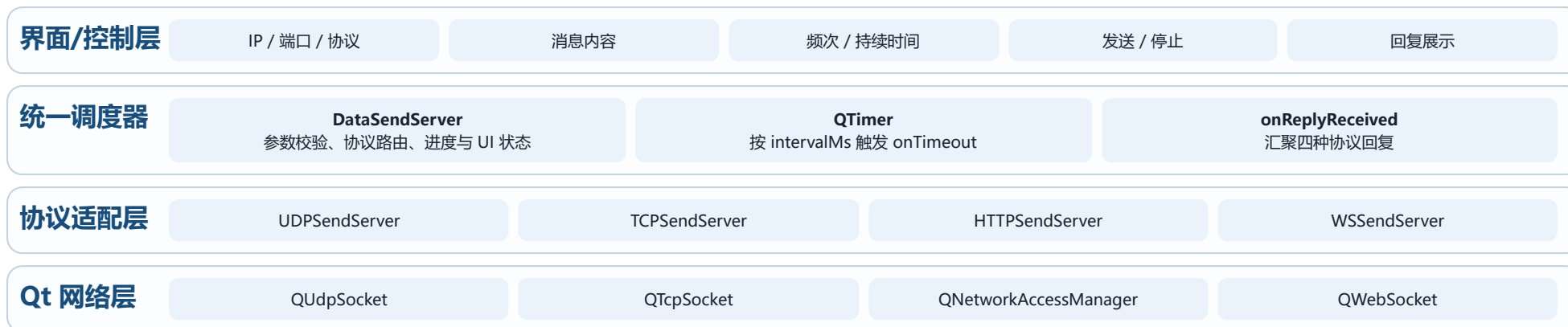
TCP 长连接

HTTP 请求/响应

WebSocket 消息

范围：整体结构与流程、协议发送/接收对应关系、各对象职责

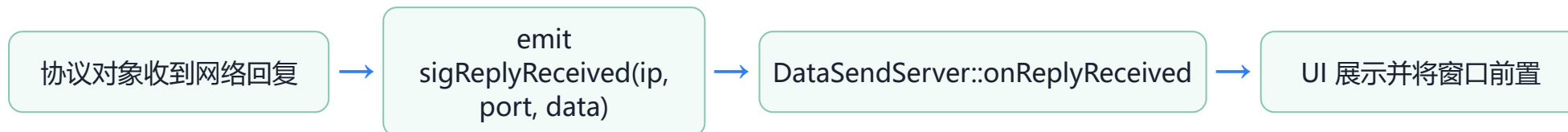
第一部分：客户端整体结构 (DataSendServer)



发送主流程

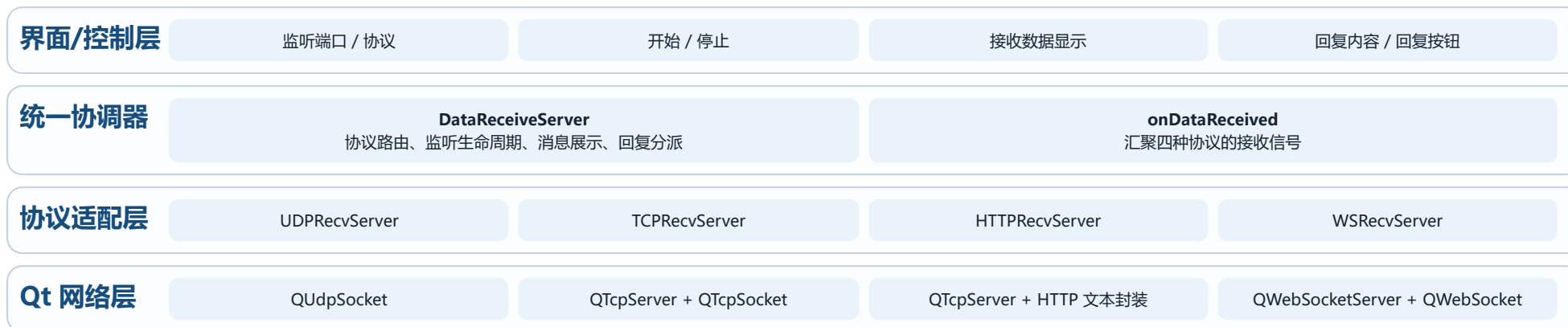


回复回流

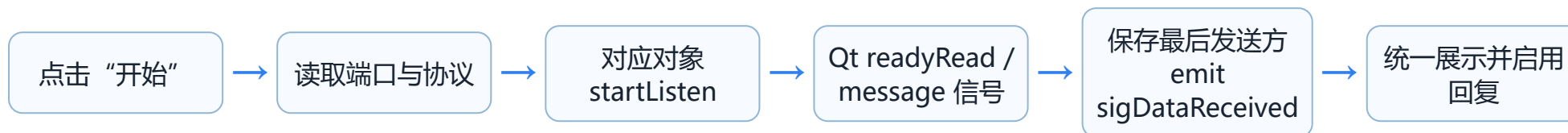


角色说明：尽管工程目录名含“Server”，DataSendServer 在业务交互中实际承担主动发起连接/请求的客户端角色。

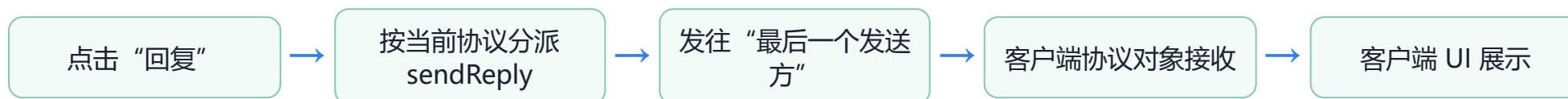
第二部分：服务端整体结构 (DataReceiveServer)



监听与接收流程

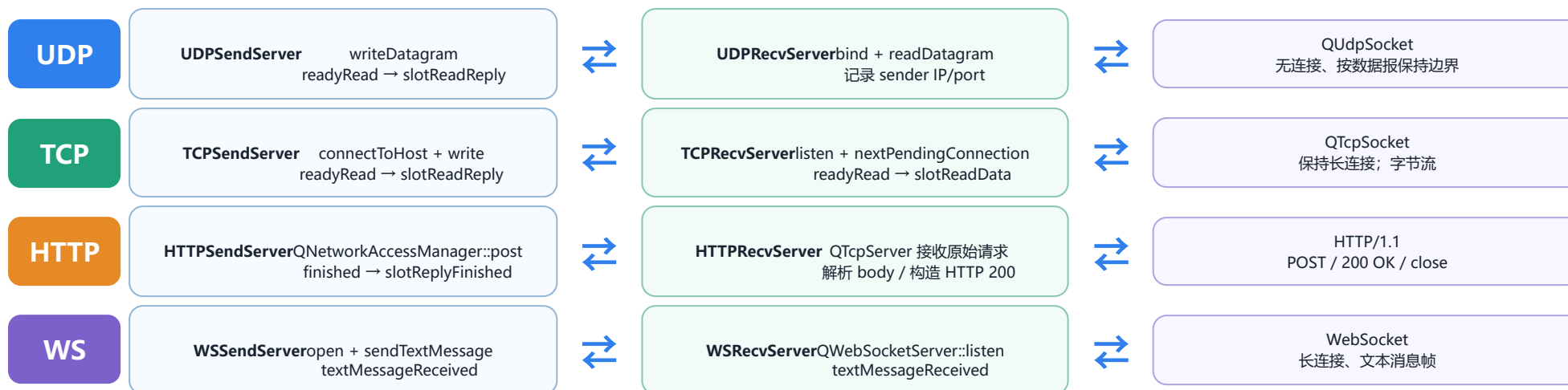


人工回复流程



回复目标策略：UDP 保存最后一次数据报的 IP/端口；TCP、HTTP、WebSocket 保存最后一次发来数据的连接对象。

第三部分：各通信协议的发送 / 接收对应关系



端到端映射

协议	正向：客户端 → 服务端	反向：服务端 → 客户端	会话 / 回复目标
UDP	sendUdpData → writeDatagram → slotReadData	sendReply → writeDatagram → slotReadReply	最后数据报的 senderIp + senderPort
TCP	sendTcpData → write → slotReadData	sendReply → write → slotReadReply	m_lastSenderSocket; 客户端保持目标连接
HTTP	sendHttpData → POST → slotReadData / parseHttpBody	makeHttpResponse(200) → finished / readAll	m_lastSenderSocket; 响应后 Connection: close
WebSocket	sendWsData → sendTextMessage → slotTextMessageReceived	sendReply → sendTextMessage → onTextMessageReceived	m_lastSenderSocket; 客户端复用同目标长连接

发送与接收协议对象职责

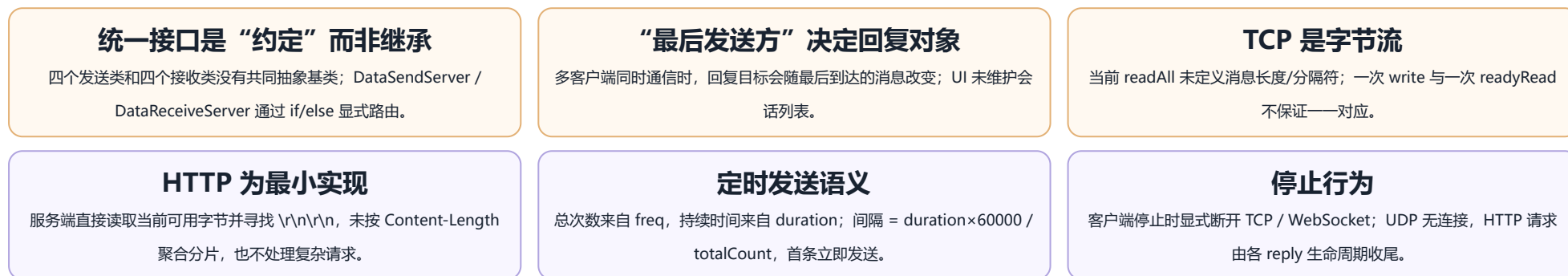
对象	核心职责	发送 / 回复	接收 / 状态管理
DataSendServer	客户端 UI 门面与统一调度	校验参数；计算发送间隔；按协议调用 send*Data	汇聚 sigReplyReceived；维护计数、进度、启停状态
UDPSendServer	UDP 客户端适配器	UTF-8 编码并 writeDatagram	绑定临时端口；循环读取回复数据报
TCPSendServer	TCP 客户端适配器	按目标建立/复用连接并 write	readyRead 读取回复；记录最后目标；支持主动断开
HTTPSendServer	HTTP 客户端适配器	构造 text/plain POST	finished 后读取响应或错误并释放 QNetworkReply
WSSendServer	WebSocket 客户端适配器	连接后发送 pendingData；已连接时直接发文本	接收文本回复；跟踪连接中/目标/断开/错误
DataReceiveServer	服务端 UI 门面与统一协调	按协议调用 sendReply	控制 start/stop；汇聚 sigDataReceived；展示消息
UDPRecvServer	UDP 监听与应答	回复最后 senderIp / senderPort	bind 端口；循环读取数据报；保存最后来源
TCPRecvServer	TCP 多连接监听与应答	向 m_lastSenderSocket write	维护 clientList；接收连接和字节流；清理断连
HTTPRecvServer	轻量 HTTP 监听与响应	构造 text/plain HTTP/1.1 200，写回后断开	基于 QTcpServer；提取首个报文头后的 body
WSRecvServer	WebSocket 多连接监听与应答	向最后发送方 sendTextMessage	维护 clientList；接收文本消息；清理断连

典型端到端时序与代码结构观察

一次“发送—接收—人工回复”时序



从源码可见的边界与注意点



依据文件：DataSendServer/DataSendServer.cpp 与四个 *SendServer；DataReceiveServer/DataReceiveServer.cpp 与四个 *RecvServer。生成日期：2026-07-13。

Modbus TCP 详细流程：从界面值到 PDU

示例：写入模式，起始寄存器 10，写入值 100,200,300。界面先生成内部命令 **WRITE 10 100 200 300**，协议类再把它转换为二进制，而不是把这段字符串直接发到网络。

功能码 0x10：写多个保持寄存器

```
pdu.append(char(0x10));
appendU16(pdu, address);
appendU16(pdu, quint16(values.size()));
pdu.append(char(values.size() * 2));
for (quint16 value : values)
    appendU16(pdu, value);
```

代码	写入字段	示例产生的字节与意义
append(char(0x10))	功能码，1字节	10: Write Multiple Registers。这里是原始字节，不是字符“1”“0”。
appendU16(address)	起始地址，2字节	10 = 0x000A → 00 0A
appendU16(values.size())	寄存器数量，2字节	3 = 0x0003 → 00 03
append(char(size*2))	数据字节数，1字节	3个寄存器 × 2字节 = 6 → 06
appendU16(value)	寄存器数据，每值2字节	100 → 00 64; 200 → 00 C8; 300 → 01 2C

得到的 PDU

10 00 0A 00 03 06 00 64 00 C8 01 2C

appendU16 使用大端序：先写高8位 value >> 8，再写低8位 value & 0xFF。例如 0x1234 被写成 12 34。

Modbus TCP 详细流程：MBAP 与完整写请求

```
QByteArray frame;
appendU16(frame, ++m_transactionId);    // Transaction ID
appendU16(frame, 0);                    // Protocol ID
appendU16(frame, quint16(1 + pdu.size())); // Length
frame.append(char(1));                    // Unit Identifier
frame.append(pdu);
```

字段	示例字节	来源与作用
Transaction ID	00 01	客户端递增的事务号；服务端原样复制到响应，用于匹配请求。
Protocol ID	00 00	Modbus TCP固定为0。
Length	00 0D	Unit ID 1字节 + PDU 12字节 = 13。
Unit ID	01	当前项目固定为1。
PDU	10 00 0A 00 03 06 00 64 00 C8 01 2C	

最终通过 `QTcpSocket::write()` 发送的完整二进制帧



QByteArray 在这里是二进制容器。char(0x10) 加入一个值为 0x10 的字节；若追加字符串 "10"，得到的会是 ASCII 字节 31 30，报文将不符合 Modbus。

Modbus TCP 详细流程：服务端反向解析与写响应

1. TCP缓存与完整帧提取



2. 解析 0x10 请求

代码逻辑	读取位置	示例结果
function = frame[7]	MBAP后第1字节	0x10
address = readU16(frame, 8)	frame[8..9]	00 0A → 10
count = readU16(frame, 10)	frame[10..11]	00 03 → 3
byteCount = frame[12]	frame[12]	06 → 后续6字节
readU16(frame, 13+i*2)	frame[13...]	100、200、300

i = 0
读 00 64 → 100
register[10] = 100

i = 1
读 00 C8 → 200
register[11] = 200

i = 2
读 01 2C → 300
register[12] = 300

3. 标准 0x10 写响应



响应只回显事务号、功能码、起始地址和写入数量，不重复返回 100/200/300。客户端若要核实际值，应再发送 0x03 读取请求。

Modbus TCP 详细流程：0x03 读取与数据转换总表

读取请求：地址10，数量3

```
00 02 00 00 00 06 01 03 00 0A 00 03
```

03为读保持寄存器；00 0A为地址10；00 03为数量3。

读取响应：值100、200、300

```
00 02 00 00 00 09 01 03 06 00 64 00 C8 01 2C
```

06表示数据区6字节；之后每2字节使用 readU16 转成一个寄存器值。

代码中的数据形态转换

阶段	示例	转换方式
界面文本	100, 200, 0x012C	按英文逗号 split，逐项 trimmed
16位数值	[100, 200, 300]	QString::toUShort (&ok, 0)；基数0支持十进制和0x十六进制
网络大端字节	00 64 00 C8 01 2C	appendU16：高字节在前、低字节在后
服务端寄存器	register[10..12]	readU16恢复为quint16并写入QVector
读取响应	00 64 → 100	客户端每2字节调用readU16，组成值列表
界面显示	[100, 200, 300]	QString::number 后用逗号连接

客户端读/写是远程 Modbus 操作，会产生请求与响应；服务端界面的读/写只操作自身 m_holdingRegisters，不产生网络报文。服务端真正的 Modbus 回复在收到客户端请求后自动构造并立即发送。

Modbus TCP 接收入口: readyRead 到 slotReadData

客户端发送请求



操作系统 TCP 接收缓冲区



QTcpSocket::readyRead



ModbusTcpServer::slotReadData

新连接建立时的准备

```
QTcpSocket* socket = m_server->nextPendingConnection();
m_clients.append(socket);
m_buffers.insert(socket, QByteArray());
connect(socket, &QTcpSocket::readyRead,
        this, &ModbusTcpServer::slotReadData);
```

动作	作用
nextPendingConnection()	取得服务端侧的客户端连接对象；真正的数据收发由该 QTcpSocket 完成。
m_clients.append(socket)	保存活动连接，停止监听时统一关闭。
m_buffers.insert(...)	为该客户端建立独立的未处理字节缓存。
连接 readyRead	数据到达时由Qt事件循环自动调用 slotReadData，不需要主动轮询。

定位本次发送数据的客户端

```
QTcpSocket* socket = qobject_cast<QTcpSocket*>(sender());
if (!socket)
    return;
```

sender() 返回本次发出 readyRead 信号的 QObject；qobject_cast 将其安全转换为 QTcpSocket。客户端A触发时得到 socketA，客户端B触发时得到 socketB，因此后续响应能回到正确连接。

slotReadData：每客户端缓存与 TCP 字节流

```
QByteArray& buffer = m_buffers[socket];
buffer.append(socket->readAll());
```



为什么 buffer 必须是引用

QByteArray& buffer 直接引用 QHash 中保存的缓存。后续 append、remove、clear 都会持久修改 m_buffers[socket]。若写成普通副本，槽函数返回后未完成报文会丢失。

为什么不能直接处理 readAll()

TCP一次交付的形态	示意	需要的处理
完整一帧	[Frame 1]	直接提取并处理
拆包	[Frame 1 前半]	保留缓存，等待后半部分
粘包	[Frame 1][Frame 2]	循环逐帧提取
一帧半	[Frame 1][Frame 2 前半]	处理Frame 1，保留Frame 2前半

TCP只保证可靠、有序的字节流，不保留Modbus报文边界。当前实现依靠MBAP的Length字段自行恢复边界。

slotReadData: MBAP Length 切分完整报文

```
while (buffer.size() >= 7)
{
    quint16 length = readU16(buffer, 4);
    int frameLength = 6 + length;
    if (length < 2 || length > 260) { buffer.clear(); return; }
    if (buffer.size() < frameLength) return;
    QByteArray frame = buffer.left(frameLength);
    buffer.remove(0, frameLength);
    ...
}
```

逐步解释

代码	动作与原因
buffer.size() >= 7	MBAP固定7字节；不足7字节时无法完整读取Length。
readU16(buffer, 4)	读取偏移4~5的大端Length字段。
frameLength = 6 + length	Length之前有Transaction ID、Protocol ID、Length自身，共6字节；Length包含Unit ID和PDU。
length < 2	至少要有1字节Unit ID和1字节功能码。
length > 260	限制异常或恶意长度，避免缓存无限增长。
buffer.size() < frameLength	完整帧尚未到齐；保留buffer并等待下一次readyRead。
buffer.left(frameLength)	复制缓存开头的一条完整Modbus TCP帧。
buffer.remove(0,...)	删除已提取帧，避免重复处理；后续字节继续留在缓存。

示例：Length = 0x000D = 13，则完整帧长度 = 6 + 13 = 19字节。

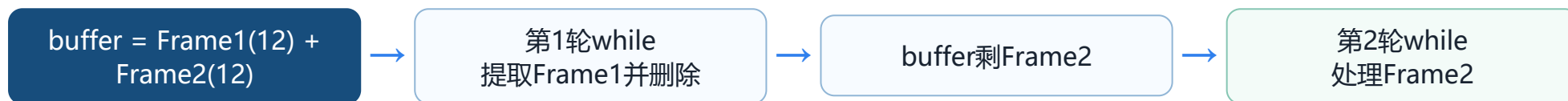
slotReadData：拆包与粘包的执行

拆包：19字节写请求分三次到达

到达批次	本次字节	缓存状态与动作
第1次	5字节	buffer=5, 小于MBAP的7字节, 不进入while, 原样保留。
第2次	再到7字节	buffer=12; 读出Length=13、frameLength=19; 12<19, 返回等待。
第3次	再到7字节	buffer=19; 提取完整19字节frame, 删除缓存前19字节并处理。



粘包：两个12字节读请求一次到达



一帧半

如果缓存是完整Frame1加Frame2前半，第一轮处理Frame1；第二轮发现Frame2长度不足，直接return。Frame2前半仍保留在对应客户端缓存，下次数据到达后继续拼接。

while循环的意义不是等待网络，而是一次性清空“当前缓存中所有完整帧”。等待下一批网络数据依靠Qt下一次readyRead事件。

slotReadData：业务处理、响应发送与界面回流

```
QString summary;  
QByteArray response = processRequest(frame, summary);  
if (!response.isEmpty())  
    socket->write(response);  
if (!summary.isEmpty())  
    emit sigDataReceived(socket->peerAddress().toString(),  
                          socket->peerPort(), summary);
```



请求类型	processRequest动作	response内容
0x03读取	读取 m_holdingRegisters	事务号、0x03、字节数、具体寄存器值
0x10写入	把请求数据写入 m_holdingRegisters	事务号、0x10、起始地址、成功写入数量
不支持功能码	不操作寄存器	功能码0x80和异常码0x01
严重不完整/协议号错误	拒绝处理	空；不会调用socket->write

两个输出通道

socket->write(response)

通过触发readyRead的同一个socket返回请求方；这是实际Modbus协议响应。

emit sigDataReceived(...)

把IP、客户端临时端口和摘要送到DataReceiveServer界面；不参与网络响应。

slotReadData负责“接收、缓存、切帧、调用处理、发送结果”；真正决定读还是写的是processRequest读取frame[7]功能码。两种响应最终共用socket->write(response)这个发送出口。

客户端完整过程（一）：界面模式解析与定时调度



模式	界面输入	校验	内部中间命令
读取	起始地址 + 读取数量	数量1~125; 地址+数量≤1000	READ 10 3
写入	起始地址 + 英文逗号分隔值	1~123个; 每值0~65535; 不能越界	WRITE 10 100 200 300

发送按钮后的调度



间隔 = 持续分钟×60000 / 总发送次数。首次请求立即发出，之后由QTimer重复触发onTimeout。读取模式形成周期轮询；写入模式会重复写相同值，工业控制命令通常应设置总次数为1。

onTimeout 的协议分派



READ/WRITE字符串只在程序内部传递。真正写入QTcpSocket的是后续生成的二进制Modbus TCP ADU。

客户端完整过程（二）：请求构造、连接与发送

READ 10 3

split空白 → 功能码03 → 地址00 0A → 数量00 03
03 00 0A 00 03

WRITE 10 100 200 300

功能码10 → 数量3 → 字节数6 → 逐值大端封装
10 00 0A 00 03 06 00 64 00 C8 01 2C

添加MBAP

事务号++

+

Protocol=0

+

Length

+

Unit ID=1

+

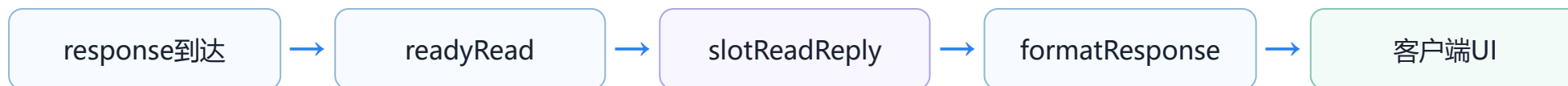
PDU

sendModbusData连接策略

当前状态	动作
端口为0	拒绝发送并提示端口无效。
已连接且目标未变化	复用现有QTcpSocket长连接。
已连接但IP/端口变化	断开旧连接，再连接新目标。
未连接	connectToHost，最多waitForConnected 3000ms。

m_socket->write(request)把完整二进制ADU加入TCP发送缓冲区；waitForBytesWritten等待交给系统网络栈。是否真正执行成功必须以服务端Modbus响应为准。

客户端完整过程（三）：响应接收、切帧与显示

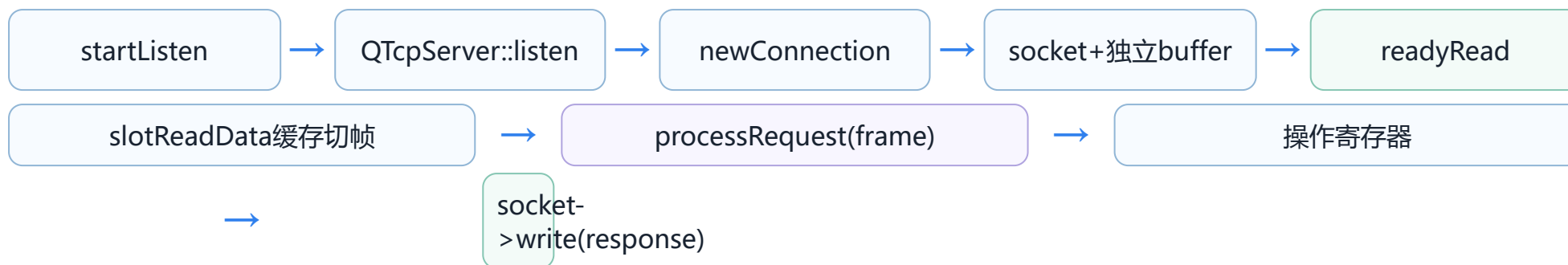


```
m_receiveBuffer.append(m_socket->readAll());
while (m_receiveBuffer.size() >= 7) {
    length = readU16(m_receiveBuffer, 4);
    frameLength = 6 + length;
    if (数据不足) return;
    frame = left(frameLength);
    remove(0, frameLength);
    formatResponse(frame);
}
```

响应功能码	解析内容	客户端显示
0x03	读取Byte Count, 从偏移9开始每2字节readU16	事务号和寄存器值列表
0x10	读取偏移8的地址、偏移10的数量	写入成功、地址和数量
最高位为1	原功能码=function&0x7F; 读取异常码	Modbus异常响应详情

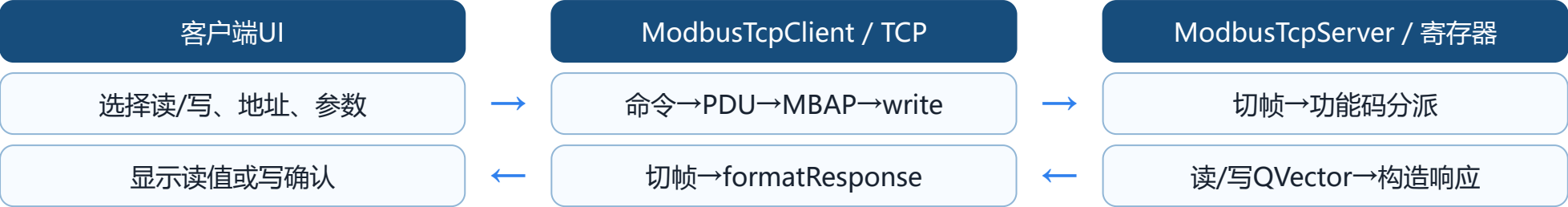
解析后发出 sigReplyReceived(ip, port, text), 由DataSendServer::onReplyReceived统一展示。事务号当前用于显示, 尚未维护并发请求匹配表。

服务端完整过程：监听、连接、切帧与自动响应



阶段	关键动作
监听	绑定Any地址和指定端口；标准端口502，项目可使用8888。
连接管理	m_clients保存socket；m_buffers为每个socket保存未完成字节；断开时移除。
切帧	读取MBAP Length；frameLength=6+Length；处理拆包、粘包和一帧半。
0x03	解析地址/数量，检查1~125和边界；读取QVector；响应携带实际值。
0x10	解析地址/数量/字节数/数据；更新QVector；响应回显地址和数量。
异常	不支持功能码→0x01；非法地址/数值→0x02或0x03；功能码最高位置1。
UI摘要	sigDataReceived只用于显示IP、客户端临时端口和摘要，不是协议响应。

完整端到端时序与服务端本地寄存器操作



服务端界面的读取/写入

服务端UI操作	实际动作	是否经过网络
读取	直接查看自身m_holdingRegisters。	否
写入	逗号拆分、toUShort转换后直接修改自身m_holdingRegisters。	否
自动响应客户端	客户端请求到达后，processRequest构造response并通过请求socket返回。	是

寄存器仅在内存

1000个quint16；服务端重启后恢复为0。

支持范围

仅Holding Register；功能码0x03和0x10。

同步等待

waitFor*可能短暂阻塞UI。

客户端负责提出远程寄存器读写请求；服务端负责保存寄存器、执行请求并自动响应；网络上传输的是二进制Modbus TCP ADU。